

Deploy: FCM + Mobile + Functions

Versione del 18/05/2026

Passi finali per portare l'app dallo stato 'web demo' a 'pubblicabile su store mobile' con push notification e backend automatizzato. Tutto il codice e' gia' scritto: questi sono i pochi click in console che mancano.

1. Web Push (VAPID key)

L'app Flutter web e' gia' configurata col pacchetto `firebase_messaging`. Manca solo la chiave VAPID per consentire le push browser.

1. Firebase Console: `console.firebase.google.com/project/silvestre-fotoservizi`
2. Project Settings (rotella in alto) -> tab Cloud Messaging
3. Sezione 'Web Push certificates' -> 'Generate key pair'
4. Copia la stringa 'Key pair' generata (e.g. BABCDEF... lunga ~88 char)
5. Apri `app/lib/services/push_notifications_service.dart`
6. Sostituisci `'static const String? webVapidKey = null;'` con la tua chiave:

```
static const String? webVapidKey = 'BABCDEF...la_tua_chiave';
```

7. Salva, rilancia il dev server.
8. Al login del cliente il browser chiederà il permesso notifiche.

2. iOS push (APNs)

Solo per build iOS. Richiede Apple Developer account.

9. Apple Developer Center -> Certificates, Identifiers & Profiles
10. Crea APNs Authentication Key (file .p8) — segui guida ufficiale
11. Firebase Console -> Project Settings -> Cloud Messaging -> Apple app configuration
12. Carica il .p8, inserisci Key ID + Team ID
13. Fine: le push iOS arrivano automaticamente.

3. Android push

Funziona automaticamente una volta che hai registrato l'app Android (sez. 5).

4. Cloud Functions deploy

[!] Richiede piano Blaze (pay-as-you-go con carta di credito).

Le 3 funzioni sono già scritte in `firebase/functions/index.js`:

- onOrderCreated -> notifica tutti gli operatori al nuovo ordine
- onOrderStatusChange -> notifica cliente al cambio stato
- scheduledPhotoCleanup -> ogni notte 03:00 cancella foto >30gg post-ritiro

Setup Blaze:

14. Firebase Console -> rotella ingranaggio -> Usage and billing -> Details and settings -> Modify plan
15. Scegli Blaze
16. Collega carta
17. Imposta Budget Alert a 5 EUR/mese in Google Cloud Console -> Billing -> Budgets

Setup Cloudinary API key per cleanup:

18. Cloudinary dashboard -> Settings -> Access Keys -> copia API Key + Secret
19. In firebase/ esegui:

```
firebase functions:secrets:set CLOUDINARY_API_KEY
```

```
firebase functions:secrets:set CLOUDINARY_API_SECRET
```

(incolli i valori quando richiesto, restano cifrati su Google Secret Manager)

Install dependencies + deploy:

```
cd firebase/functions
```

```
npm install
```

```
cd ..
```

```
firebase deploy --only functions --token "$FIREBASE_TOKEN"
```

Verifica:

20. Firebase Console -> Functions -> dovresti vedere 3 funzioni 'deployed'
21. Crea un ordine test dall'app cliente
22. Apri Logs della funzione onOrderCreated -> vedi 'New order ...' loggato

5. Registrazione app Android in Firebase

23. Firebase Console -> Project Settings -> tab General -> scroll a 'Your apps'
24. Icona Android -> Add app
25. Package name: com.silvestrefotoservizi.app
26. App nickname: 'Silvestre Android'
27. SHA-1: opzionale per ora (lascia vuoto). Necessario per Google Sign-In.
28. Register app -> scarica google-services.json

29. Sposta il file in: SilvestreApp/app/android/app/google-services.json

30. Apri app/android/app/build.gradle e aggiungi in fondo:

```
apply plugin: 'com.google.gms.google-services'
```

31. Apri app/android/build.gradle, in plugins{} aggiungi:

```
id "com.google.gms.google-services" version "4.4.2" apply false
```

6. Registrazione app iOS in Firebase

32. Firebase Console -> Project Settings -> tab General -> Add app -> iOS

33. Bundle ID: com.silvestrefotoservizi.app

34. App nickname: 'Silvestre iOS'

35. Register app -> scarica GoogleService-Info.plist

36. Sposta in: SilvestreApp/app/ios/Runner/GoogleService-Info.plist

37. Apri il file ios/Runner.xcworkspace in Xcode (sul tuo Mac)

38. Drag-and-drop GoogleService-Info.plist sotto il gruppo Runner

39. Selezionalo, in inspector destra spunta 'Runner' target

7. Rigenera firebase_options.dart con tutte le piattaforme

Dopo aver registrato Android e iOS in Console, lancia FlutterFire CLI per regenerare automaticamente:

```
dart pub global activate flutterfire_cli
```

```
cd app
```

```
flutterfire configure --project=silvestre-fotoservizi
```

Il comando rigenera lib/firebase_options.dart con le 3 piattaforme corrette (web + android + iOS).

8. Checklist finale

☐ VAPID key web sostituita in push_notifications_service.dart

☐ Apple Developer attivo (99 EUR/anno) e APNs .p8 caricato in Firebase

☐ App Android registrata in Firebase + google-services.json in app/android/app/

☐ App iOS registrata in Firebase + GoogleService-Info.plist in app/ios/Runner/

☐ flutterfire configure eseguito (firebase_options.dart aggiornato per 3 piattaforme)

- [] Blaze plan attivato + Budget Alert 5 EUR/mese
- [] Cloudinary API Key + Secret in Firebase Secrets
- [] firebase deploy --only functions completato senza errori
- [] Test: crea ordine cliente -> arriva push operatore
- [] Test: operatore cambia stato -> arriva push cliente
- [] Apple App Store Connect creato (per build iOS)
- [] Google Play Console creato (per build Android)